Welcome to our new Tech Talk

SW ENGINEERING
VIVA ENGAGE PAGE
JOIN NOW

SW ENGINEERING
SHAREPOINT PAGE
JOIN NOW

SW ENGINEERING
MAGAZINES PAGE
JOIN NOW

Capgemini engineering

SOFTWARE ENGINEERING COMMUNITY

Agentic SW Engineering with Claude Code

Thomas Remy
Head of EMEA South
at Anthropic

Arnaud Doko
Applied AI Architect
at Anthropic

Jorge Lopes
ER&D Software IDA

Luís Alves
ER&D Software Product
Sub-Practice IDA

2026/02/11

Capgemini engineering

Agenda

01	Anthropic presentation & Claude Code demo
02	Claude Code Setup in Capgemini Environment
03	Q&A

Meeting resources

Claude Code Playbook

Slides presented by Anthropic

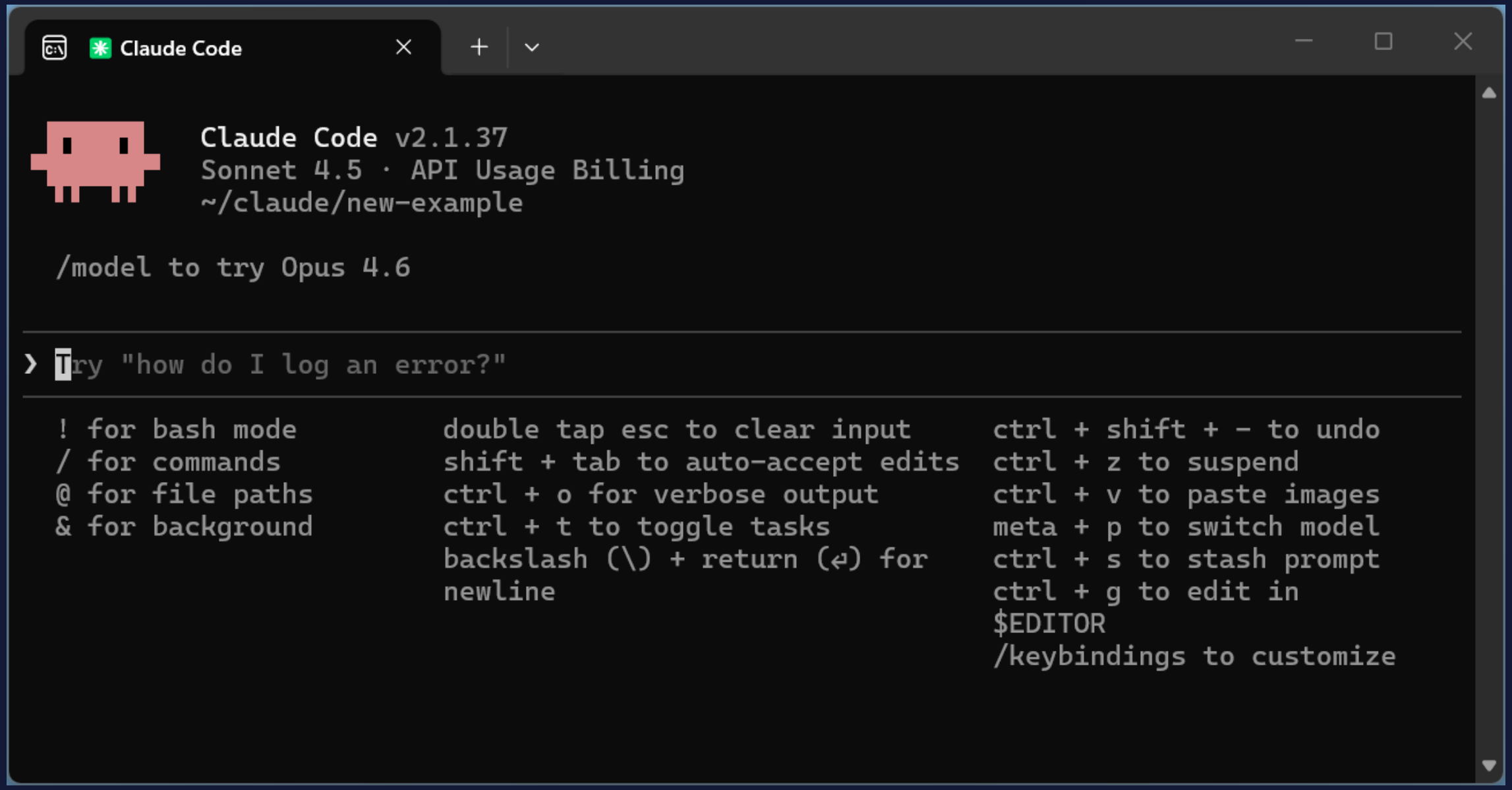
Anthropic Speaker notes

Teams Meeting recap (video + transcript)

Meeting recording (video only)

Tech Talk Q&A

1. Anthropic presentation & Claude Code demo

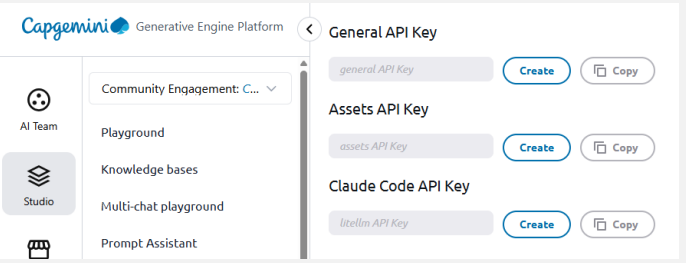

Claude Code
Claude Code v2.1.37
Sonnet 4.5 · API Usage Billing
~/claude/new-example
/model to try Opus 4.6
› Try "how do I log an error?"
! for bash mode
/ for commands
@ for file paths
& for background
double tap esc to clear input
shift + tab to auto-accept edits
ctrl + o for verbose output
ctrl + t to toggle tasks
backslash (\) + return (↵) for newline
ctrl + shift + - to undo
ctrl + z to suspend
ctrl + v to paste images
meta + p to switch model
ctrl + s to stash prompt
ctrl + g to edit in $EDITOR
/keybindings to customize

2. Claude Code Setup in Capgemini Environment

Claude Code is available through Generative Engine (GEP)

What is GEP Providing?

Generative Engine Platform now provides a specialised API endpoint so you can connect Claude Code to Anthropic LLMs provided by GEP.



How to Get Started

1. **Get a Collaborative Studio, either in GEP US Hosted or GEP UE Hosted: GEP's Claude Code API is a paid feature exclusively available in a GEP Collaborative Studio.**

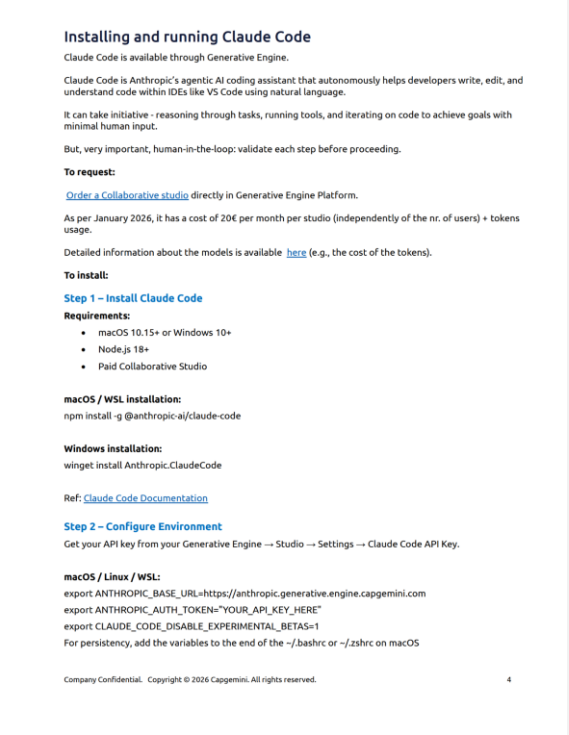
   As per January 2026, it has a cost of 20€ per month per studio (independently of the nr. of users) + usage. Detailed usage pricing information available here.

   Note: the API keys for Claude Code must match the instance in use (US or UE). Request them in the right instance.

2. **Install Claude Code CLI & Configure your Claude Code environment using your GEP Claude Code API key**

   Follow these instructions (snapshot on the right side of this slide)

3. **Run Claude Code!**



Installing and running Claude Code

Claude Code is available through Generative Engine.

Claude Code is Anthropic's agentic AI coding assistant that autonomously helps developers write, edit, and understand code within IDEs like VS Code using natural language.

It can take initiative - reasoning through tasks, running tools, and iterating on code to achieve goals with minimal human input.

But, very important, human-in-the-loop: validate each step before proceeding.

To request:

Order a Collaborative studio directly in Generative Engine Platform.

As per January 2026, it has a cost of 20€ per month per studio (independently of the nr. of users) + tokens usage.

Detailed information about the models is available here (e.g., the cost of the tokens).

To install:

Step 1 – Install Claude Code

Requirements:

- macOS 10.15+ or Windows 10+
- Node.js 18+
- Paid Collaborative Studio

macOS / WSL installation:

npm install -g @anthropic-ai/claude-code

Windows installation:

winget install Anthropic.ClaudeCode

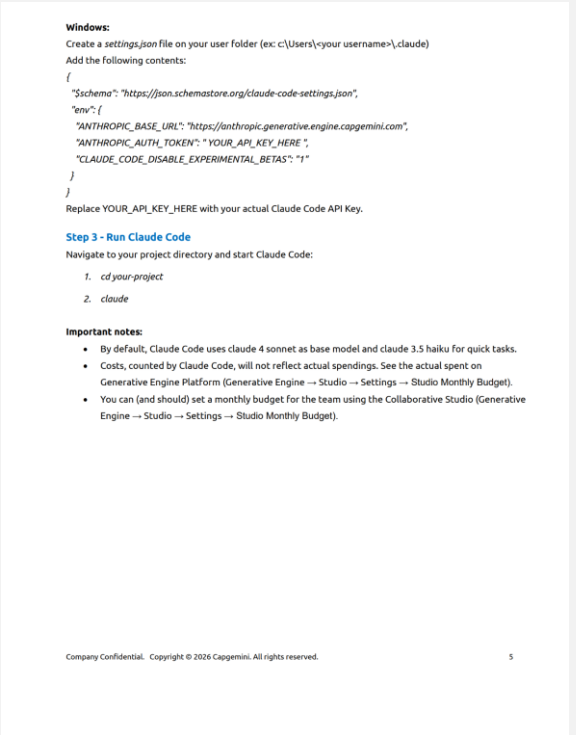
Ref: Claude Code Documentation

Step 2 – Configure Environment

Get your API key from your Generative Engine → Studio → Settings → Claude Code API Key.

macOS / Linux / WSL:

export ANTHROPIC_BASE_URL=https://anthropic.generative.engine.capgemini.com
export ANTHROPIC_AUTH_TOKEN="YOUR_API_KEY_HERE"
export CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS=1
For persistency, add the variables to the end of the ~/.bashrc or ~/.zshrc on macOS



Windows:

Create a *settings.json* file on your user folder (ex: c:\Users\<your username>\.claude)
Add the following contents:

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "env": {
    "ANTHROPIC_BASE_URL": "https://anthropic.generative.engine.capgemini.com",
    "ANTHROPIC_AUTH_TOKEN": " YOUR_API_KEY_HERE ",
    "CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS": "1"
  }
}
```

Replace YOUR_API_KEY_HERE with your actual Claude Code API Key.

Step 3 - Run Claude Code

Navigate to your project directory and start Claude Code:

1. *cd your-project*
2. *claude*

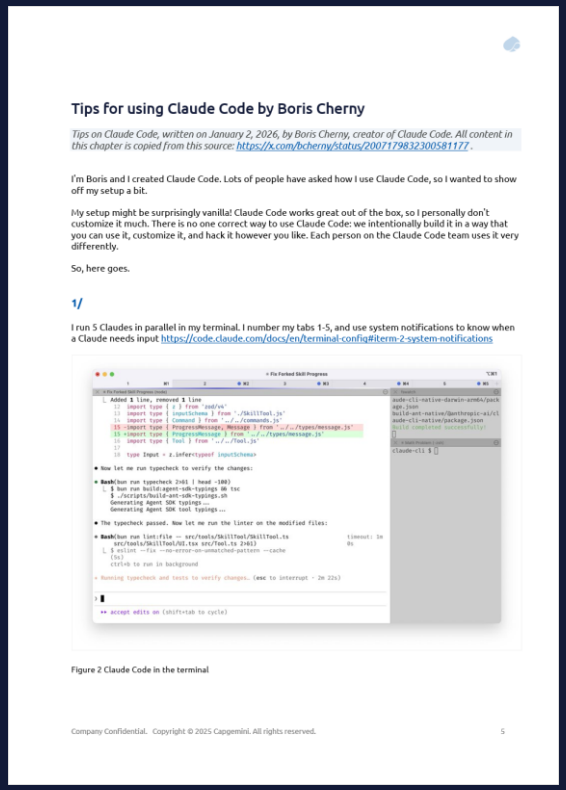
Important notes:

- By default, Claude Code uses claude 4 sonnet as base model and claude 3.5 haiku for quick tasks.
- Costs, counted by Claude Code, will not reflect actual spendings. See the actual spent on Generative Engine Platform (Generative Engine → Studio → Settings → Studio Monthly Budget).
- You can (and should) set a monthly budget for the team using the Collaborative Studio (Generative Engine → Studio → Settings → Studio Monthly Budget).

Need More Help? *Ask in the GEP Developer Community or raise a GEP i4u Support Ticket here*

Using Claude Code

Check the Claude Code Playbook for tips about using Claude Code



Tips for using Claude Code by Boris Cherny

Tips on Claude Code, written on January 2, 2026, by Boris Cherny, creator of Claude Code. All content in this chapter is copied from this source: https://x.com/bcherny/status/2007179832300581177.

I'm Boris and I created Claude Code. Lots of people have asked how I use Claude Code, so I wanted to show off my setup a bit.

My setup might be surprisingly vanilla! Claude Code works great out of the box, so I personally don't customize it much. There is no one correct way to use Claude Code: we intentionally build it in a way that you can use it, customize it, and hack it however you like. Each person on the Claude Code team uses it very differently.

So, here goes.

1/

I run 5 Claudes in parallel in my terminal. I number my tabs 1-5, and use system notifications to know when a Claude needs input https://code.claude.com/docs/en/terminal-config#iterm-2-system-notifications

Figure 2 Claude Code in the terminal

Cheat sheet

Created from the above tips (source).

CLAUDE CODE

Top 10 Production Tips from Boris

Parallelization Strategy 1

Run 5 terminal instances + 5-10 web sessions simultaneously. Use numbered tabs and system notifications for orchestration. Leverage `&` handoffs and `--teleport` for session migration.

Model Selection 2

Opus 4.5 with thinking exclusively. Superior tool use and reduced steering requirements outweigh latency costs—net faster iteration despite model size.

Shared Context Files 3

Team-maintained `CLAUDE.md` in git. Document failure modes immediately. Update multiple times weekly to compound model performance across iterations.

PR Review Automation 4

`@.claude` tags in PR comments trigger Github action to update `CLAUDE.md`. Implements compounding engineering at code review stage.

Plan Mode First 5

Start sessions in Plan mode (⇧⇥ twice). Iterate until plan approval, then switch to auto-accept. Quality planning enables single-shot execution.

Slash Command Library 6

Inner-loop workflows as commands in `.claude/commands/`. Example: `/commit-push-pr` with inline bash pre-computation eliminates model latency.

Subagent Workflows 7

Deploy specialized subagents: code-simplifier for post-processing, verify-app for E2E testing. Automate repetitive PR workflows.

PostToolUse Hooks 8

Automatic formatting via hooks handles final 10% polish. Prevents CI failures from stylistic inconsistencies.

Permission Management 9

`/permissions` pre-allowlisting over `--dangerously-skip-permissions`. Environment-specific safe commands in `.claude/settings.json`, git-tracked and team-shared.

Verification Loops 10

Critical: provide Claude self-verification mechanisms. 2-3x quality improvement. Examples: test suites, Chrome extension UI testing, domain-specific validation scripts.

Figure 13 Claude code cheat sheet

Q&A

This section contains a selection of questions from the same thread, answered by Boris Cherny.

What's the best way to create quality validation loops? Or to learn how to create them? I know that's a nuanced question "it depends" but I'm more interested in learning how to think about building them, if that makes sense.

It's really simple actually, I think people sometimes over-complicate it.

1. Give Claude a tool to see the output of the code: if server code, a way to start the server/service; if web code, a way to see and interact with the UI; etc.

2. Tell Claude about the tool: this is just tuning the tool descriptions so Claude understands when it should use the tool

That's literally it. Claude will figure out the rest

How do you get Claude to be able to view the web browser? Claude Code tells me it cannot do that.

claude --chrome

How big is your Claude markdown file, and is it used for coding style, general coding principles, specific ways you work, commands, or other stuff?

Our checked in http://CLAUDE.md is 2.5k tokens. It covers:

- common bash commands
- code style conventions
- ui and content design guidelines
- how to do state management, logging, error handling, gating, and debugging
- pull request template

For each of your Claude instances, do you run one single feature at a time? For example, I find myself doing multiple features in the same context window, then it'll compact etc, and I'd continue (Partially because I get carried away). Secondly, do you have a certain way to avoid overlap when working on the same file if running multiple Claude agents?

Yes, one at a time. Each agent gets its own git checkout.

How do you handle multiple features in parallel, since you said you use 5 tabs?

I should have clarified above, I use 5 separate git checkouts of the same repo.

So, you are using 10 different git clones of the repo with a different branch for each? As in, you have the repo cloned 10 times?

DO's and DON'Ts

Review & Test Everything: Manually verify all generated code against project requirements and standards. Never merge without human-in-the-loop validation.

Sanitize All Prompts: Strip personal identifiable information, client data, and secrets before prompting. Put sensitive data outside the repository visible to Claude Code.

Run Security Scans: Use automated tools to detect vulnerabilities and hardcoded secrets in AI output. Verify licenses for any new third-party libraries.

Don't Run Blindly: Never execute commands or logic you don't fully understand. Agentic tools can perform destructive actions if unmonitored.

Don't Delegate Security: Do not let AI design custom encryption or auth protocols. Use human-vetted, industry-standard security patterns.

Don't Assume Authorization: Never use AI on a new project without explicit client consent and Engagement Manager approval.

Check the Claude Code Playbook for full list

Q & A

SHARE YOUR FEEDBACK

Capgemini engineering